Lesson 1: Databases, tables, keys and relationships

Phase 1 – SQL | 11 November 2026 | Practice database: business_db (Karoo Industrial Supplies)

1. Goal

By the end of this lesson you can:

- say what a database, table, row and column are
- spot the primary key and foreign keys in a table
- trace how tables connect, so you know which ones a question needs

2. Concept

Term	Meaning
Database	An organised store of data. Yours is `business_db`.
Table	One kind of thing, laid out as a grid. `customers` holds customers and `products` holds products.
Row (record)	One item in the table. One customer is one row.
Column (field)	One fact about every item, such as `city` or `unit_price`. Every column has a data type: text, whole number, decimal, date, true/false.
Primary key (PK)	A column whose value is different for every row and never empty. It identifies one row, the way a staff number identifies one employee. Names are a poor key because two customers can share one.
Foreign key (FK)	A column that holds the primary key of a row in another table. It is the link that connects tables.

A small example in the style of the book, using two tables:

supplier_id (PK)	supplier_name
1	Cape Fans
2	Atlas Motors

part_id (PK)	part_name	supplier_id (FK)
10	Fan blade	1
11	Motor 5 kW	2
12	Fan shaft	1

`parts.supplier_id` is a foreign key pointing at `suppliers.supplier_id`. The supplier's name is stored **once**, not repeated on every part. If "Cape Fans" changes its name, you fix one row.

Relationships come in three kinds

- **One-to-many:** one supplier supplies many parts, but each part has one supplier. The foreign key sits on the "many" side. This is the most common kind.
- **Many-to-many:** one order can contain many products, and one product can appear on many orders. A database handles this with a **link table** in the middle (an order-items table) that

holds both keys.

- **Self-reference:** a table points at itself, such as an employee's manager, who is also an employee.

3. Real-world example: our database

Karoo Industrial Supplies has seven tables.

Table	Primary key	Foreign keys
employees	employee_id	manager_id → employees (self-reference)
customers	customer_id	none
products	product_id	none
orders	order_id	customer_id → customers, employee_id → employees
order_items	order_item_id	order_id → orders, product_id → products
sales_targets	target_id	employee_id → employees
production_log	log_id	product_id → products

Reading the map. Suppose your manager asks: "Which company bought the most spare parts last quarter?" The company is in customers. The parts are in products. Neither table knows about the other. You connect them by following the keys:

> **customers → orders → order_items → products**

order_items is the link table that makes the many-to-many relationship between orders and products work. Once you can trace a path like this, you know which tables a question needs before you write any SQL. That is most of the skill in joins, which come in Lesson 6.

Looking at the data in pgAdmin

- In pgAdmin, expand business_db → **Schemas** → **public** → **Tables**.
- Right-click orders → **View/Edit Data** → **First 100 Rows**.
- Look at the columns. shipped_date is empty for some rows. That is a **NULL**, meaning "no value". We cover NULLs in Lesson 3.

To see a table's column names and types, open the **Query Tool** on business_db and run this (you will learn SELECT properly in Lesson 2, so just copy it for now):

```
SELECT column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_name = 'orders'
ORDER BY ordinal_position;
```

4. Common mistakes

- **Treating a name as a key.** Two customers can share a name or a city. Use the ID column to identify a row.
- **Mixing up which side holds the foreign key.** It always sits on the "many" side. One customer has many orders, so customer_id is in orders, not the other way round.
- **Assuming that columns with the same name are the same thing.** unit_price is in both products and order_items. The first is today's list price and the second is the price charged on that order, so they can differ.

- **Skipping the link table** in a many-to-many path. You cannot jump from `orders` straight to `products`.
- **Thinking NULL means zero or blank text.** NULL means "no value recorded", which is different from 0.

5. Practice

Write your answers in the chat. For Exercises 1 and 2 you do not need SQL. Exercise 3 uses a short query.

Exercise 1 (keys)

For `sales_targets` and `production_log`, state the primary key and each foreign key, and which table each one points to. Then say in one sentence what a single row in each table represents.

Exercise 2 (tracing a path)

Using only the relationship map above, write the chain of tables you would follow to find **the company name of every customer who bought a "Float Valve 25 mm"**. Then name which table is the link table in that chain, and say why it is needed.

Exercise 3 (explore)

In the pgAdmin Query Tool, run:

```
SELECT employee_id, first_name, last_name, job_title, manager_id
FROM employees;
```

Then answer:

- a) Which employee has no value in `manager_id`, and what does that tell you?
- b) Who reports directly to the Operations Manager (`employee_id` 3)? Name them.
- c) Is the relationship between `employees` and `orders` one-to-many or many-to-many? Explain in one sentence.

> **My answers** (use the space below or the chat). Send them to Claude for review, then say **next** for the portfolio files.